



۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی



نوت ۱۵ مهر

ادامه فصل ۱ اسلاید ۱۰۸ تا ۱۲۸ [۱]

فلسفه طراحی اینترنت

مفاهیم کلیدی

Naming:

با روش‌های نام‌دهی، می‌توان ساختار دستگاه‌های مختلف و یا سرویس‌ها و پروتکل‌ها را مشخص کرد و به آن‌ها شناسه مشخص داد.

Layering:

معماری لایه‌بندی از هر لایه به ما *abstraction* می‌دهد که احتمالا اجازه می‌دهد به طور مستقل از هم و بدون اطلاع دقیق از پیاده‌سازی‌ها، پروتکل‌های هر لایه پیاده‌سازی شوند. در مورد *abstraction*، لایه‌ها کافی‌ست طبق یک *interface* خاصی با هم ارتباط برقرار کنند و لازم نیست نگران پیاده‌سازی دیگر لایه‌ها باشند. چون لایه‌ها بدون اینکه لایه‌های دیگر را مختل کنند، می‌توانند تغییر کنند.

Protocols:

پروتکل هر لایه یک زبان مشترک بین دستگاه‌های مربوط به آن لایه ایجاد می‌کند. در پروتکل‌ها باید سینتکس و مفهوم (*semantic*) هر عبارت را مشخص کنیم.

Resource allocation: برای پهنای باند لینک و حافظه و مسیرها (لینک)‌های موجود در شبکه، به الگوریتم‌های تخصیص منابع نیاز داریم.

نقش استاندارد سازی

می‌دانیم که استاندارد های مختلفی برای معماری های مختلف شبکه وجود دارد، مثل *client server* یا معماری *peer to peer*.

معماری لایه بندی هم همانطور که گفتیم، به سازماندهی و ساختاردهی شبکه‌های پیچیده کمک می‌کند. (اما هنوز این موضوع که یک عملکرد یکسان در دو لایه پیاده‌سازی شود، از معایب آن است)

مفهوم *Unified Internet Layering* هم به این اشاره که می‌کند که لایه‌های مختلف (مثلا لایه اپلیکیشن با دیگر لایه) چگونه و طبق چه استانداردهایی باید بین هم ارتباط برقرار کنند و این مستقل از تکنولوژی‌های مورد استفاده در هر لایه است.

مفهوم $E2E = end\ to\ end$ در شبکه، باعث می‌شود که بسیاری از کارها در *host*ها که در انتهای شبکه قرار دارند انجام شود و نه در میانه شبکه یا همان *core* شبکه. این باعث می‌شود که شبکه ساده شود و با همان پروتکل *IP* کل کارش را انجام دهد و پیچیدگی خاصی نداشته باشد. این موضوع خیلی به سود *commercial reality*ها که شبکه را مدیریت می‌کردند، تمام شد، چون کارشان راحت‌تر شد.

انواع سطوح *abstraction* توسط لایه‌بندی:

- سطح انتزاع اپلیکیشن (لایه ۵)
- سطح انتزاع اپلیکیشن یک سمت به اپلیکیشن سمت دیگر که مثل کانال مجازی می‌شود. (لایه ۴)
- سطح انتزاع *Host* مبدا به *Host* مقصد (لایه ۳)
- سطح انتزاع سخت‌افزار لینک و ارتباط دو دستگاه مجاور (لایه ۲ یا لایه ۱ و ۲)

اصل *End to End*

پروتکل‌ها بایب به گونه ای باشند که در نقاط انتهایی شبکه *end points* بتوانند تعریف و پیاده‌سازی شوند و قابل برنامه‌نویسی باشند. به این شکل، هر کس در هر زمان می‌تواند خودش سرویس‌های جدیدی، به شبکه اضافه کند.

پس بعضی از عملکردها (*Functionality*) ی شبکه را می‌توان در *host* پایانی پیاده‌سازی کرد، چون اینگونه بدون کمک شبکه می‌تواند پیاده‌سازی صورت گیرد و از طرفی خود شبکه هم قابل اتکا *reliable* نیست.

انواع پیاده‌سازی *functionality*های شبکه

- راه اول: درگیر کردن گره‌های شبکه، یعنی تک تک گره‌ها از مبدا تا مقصد، به طور قابل اتکا (*reliable*)، آن *functionality* را پیاده‌سازی کنند تا یک مسیر قابل اتکا از ابتدا تا انتها داشته باشیم
- راه حل دوم: پیاده‌سازی و آپدیت عملکردها در انتهای شبکه رخ دهد و تنها در صورت گریزناپذیر بودن، گره‌های شبکه را نیز درگیر کنیم.

مقایسه انواع پیاده‌سازی‌های عملکرد

در مورد راه اول: اول اینکه ممکن است یکی از گره‌های میانی خراب شود و کار کلاً خراب شود. به هر حال گیرنده باید صحت دیتا رو دوباره بررسی کند چون اعتمادی به گره‌ها نیست.

پیاده‌سازی عملکرد در شبکه، پیچیدگی پیاده‌سازی‌اش را کاهش نمی‌دهد و بلکه افزایش نیز می‌دهد. پیچیده شدن شبکه باعث تحمیل *overhead* و تاخیر به همه اپلیکیشن‌ها می‌شود حتی اگر به آن عملکرد نیاز نباشد.

فقط در بعضی از شرایط محدود، میتواند اینکار مفید باشد. مثلاً به قول استاد، اگر لینک‌های شبکه پر اتلاف باشد و پکت‌های زیادی را *drop* کند، احتمالاً پیاده‌سازی عملکرد در شبکه، می‌تواند به *error correcting* به صورت محلی و کنترل و مدیریت این موضوع کمک کند.

راه حل دوم کامل است، چون عملکرد در لایه اپلیکیشن در *host* مبدا و مقصد انجام شده و لایه‌های دیگر و هسته شبکه درگیر نشده.

اگر قرار باشد لایه‌های پایینی درگیر شود، صرفاً برای کارآمدتر کردن سیستم و اپلیکیشن است.

انواع تفاسیر از *E2E*

تفسیر محافظه‌کارانه *conservative*

اینکه تمام لایه‌ها و دستگاه‌های میانی در پیاده‌سازی عملکرد دخیل هستند. اگر بطور کامل مطمئن باشیم که همه نیازمندی‌های یک عملکرد در یک لایه قابل پیاده‌سازی است، در همان لایه پیاده‌سازی می‌کنیم و لایه‌های پایینی را درگیر نمی‌کنیم اما برای اینکه ایمن‌تر و مطمئن‌تر باشیم، بهتر است بعضی از توابع آن عملکرد را در لایه‌های پایینی و یا دستگاه‌های میانی پیاده‌سازی کنیم. بهتر و ساده‌تر این است که *host* هم درگیر پیاده‌سازی عملکرد بشود.

تفسیر رادیکال

این تفسیر دقیقاً مقابل تفسیر محافظه‌کارانه است. اینکه فقط از همان *host* مبدا و مقصد درگیر پیاده‌سازی عملکرد هستند. یعنی اگر به طور کامل توسط *host*‌ها قابل پیاده‌سازی است، لازم نیست از پیاده‌سازی در شبکه استفاده کنیم. این مورد باعث سادگی لایه‌های پایینی شبکه می‌شود و همین موضوع، انعطاف پذیری شبکه را افزایش می‌دهد.

تفسیر میانه *moderate*

عملکرد تا حد ممکن در *host*‌ها پیاده‌سازی شود اما اگر کارایی سیستم (نسبت به حالتی که به طور کامل در *host* پیاده‌سازی شود) بتواند افزایش پیدا کند، بهتر است از لایه‌های پایینی شبکه و *device*‌های میانی هم کمک بگیریم.

اما این پیاده‌سازی عملکرد نباید به گونه‌ای باشد که *overhead* و تاخیر اضافی به اپلیکیشن‌هایی که از آن عملکرد استفاده نمی‌کند، تحمیل کند.

مفهوم *fate sharing*

این مفهوم، به نوعی مفهومی مرتبط با *E2E* است. حالت‌ها *state* را بهتر است در مکانی ذخیره کنیم که موجودیت متکی به آن حالت‌ها قرار دارند. چون حالت‌ها را برای آن موجودیت نگه می‌داریم، بهتر است در همان مکان موجودیت ذخیره شود. این موضوع به این دلیل است که مثلاً اگر شما حالت را در یک *host* موجودیت آن حالت ذخیره کنید، اگر *host* خراب شود، طبعاً موجودیت و حالت هم از بین می‌رود و این موضوع دیگر برای ما اهمیت ندارد چون خود موجودیت هم کار نمی‌کند. اما فرض کنید که حالتش را در روتر میانی مسیر مبدا تا مقصد ذخیره کرده باشیم. اگر روتر میانی خراب شود، حالت از بین می‌رود، در صورتی که می‌دانیم *host*‌ها هنوز سالم هستند و می‌توانستند از مسیر دیگر با هم ارتباط برقرار کنند. اینگونه چون حالت از بین رفته، دیگر نمی‌توانند مثل قبل به ارتباط خود ادامه دهند. پس هر جا که موجودیت است، حالت آن را در همان موجودیت نگه می‌داریم. و چون موجودیت‌ها و اپلیکیشن‌های وابسته به حالت‌ها، بیشتر در *host*‌ها قرار دارند، بهتر است این حالت‌ها را در *host*‌های پایانی نگه داریم. چون خرابی هر اپلیکیشن، به خودش مربوط است و نه دیگران. پس *state* را در همانجا نگه می‌داریم. اگر به یاد داشته باشید، در روش *circuit switching*، حالت‌ها در روترهای میانی نگه داشته می‌شد. اما در *packet switching*، در همان *host*‌های انتهایی. در پروتکل‌هایی همچون *HTTP* و *NFS* هم، حالت‌ها در *Host*‌ها نگهداری می‌شوند.

معماری آینده اینترنت (*FIA = Future Internet Architecture*)

چهار گروه روی آینده اینترنت کار می‌کنند:

- بخش *Named data networking*: این بخش روی ارائه محتوا به سریع‌ترین حالت ممکن کار می‌کنند، برعکس اینترنت فعلی که باید روی آدرس سایت کلیک کنی تا محتوا را ببینی.
- بخش *Mobility fast*: این گروه روی گره‌های سیار کار می‌کنند (حدس می‌زنم در شبکه‌های بی‌سیم) چون در اینترنت عادی، گره‌های میانی به صورت ثابت فرض شده و سیار بودنش به عنوان اصل مدنظر قرار نگرفته.
- بخش *Nebula*: این گروه سعی می‌کند از *cloud* در معماری اینترنت کمک می‌گیرند
- بخش *XIA = eXpressive Internet Architecture*: این بخش سعی می‌کند مدل‌های متنوع شبکه را ارائه دهد (تنوع نقاط پایانی *end - point* رو ساپورت کند و امنیت مربوط به آنها را تضمین کند). همچنین

ارتباط قابل اعتماد بین عناصر شبکه را برقرار می‌کند و همچنین، بین ذی‌نفعان آن، (مثلا ارائه دهندگان اینترنت) هماهنگی برقرار می‌کند.

شبکه‌های مبتنی بر محتوا $CCN = Content Centric Network$

در این مدل شبکه، که از انواع مدل‌های آتی شبکه می‌تواند باشد، هر کدام از دستگاه‌های میانی، در خود *cache* دارند. پس در مسیر، هر کدام که محتوای خواسته شده در بسته را داشته باشد (به بسته، *interest* نیز می‌گویند)، محتوای خواسته شده را خودش برمی‌گرداند.

در این نوع شبکه‌ها، از آنجا که مفهوم شناسه یا *identity* به طور مشخص وجود ندارد، با مشکلات متعددی همچون *spam* یا هزریات، *spoofing* یا جعل هویت، *denial of service* و *route hijacking* یا ربایش مسیر ممکن است مواجه شویم.

احتمالا اگر در شبکه *trust* وجود داشت، هیچ‌کدام از این اتفاقات رخ نمیداد.

بعضی از پروتکل‌ها با این فرض ساخته شده که بین کاربران، اعتماد متقابل وجود داشته باشند. یعنی کاربران دیگر لازم نیست نگران *spoofing* در آدرس *MAC* و یا *IP* باشند و یا *spam* دریافت یا تولید کنند. هر پورت به اپلیکیشنی متناظر شده است و اینطور نیست که اعداد پورت‌ها دلخواه یا بی‌معنی باشد.

مشخصات پروتکل‌ها به صورت *public* است و کاربران با همکاری هم، می‌توانند روی پیاده‌سازی آن کار کنند. پس همه باید به پروتکل و لازمه‌های آن پایبند باشند.

چالش‌های شبکه

چالش‌های زیادی می‌تواند در شبکه به وجود بیاید، مخصوصا زمانی که فرض کردیم *trust* داریم:

- می‌دانیم که شبکه، به سیستم توزیع شده است و صاحب و مسئول یکتایی ندارد. می‌دانیم که ترافیک از چندین سیستم خودمختار عبور پیدا می‌کند $AS = Autonomous System$. حال اگر اشتباهی رخ دهد، نمی‌توان مسئولیت را به گردن کس خاصی بندازیم.

- اگر شبکه بخواهد ارتقا پیدا کند، چطور باید این کار را هندل کنیم؟

- اگر فرض کرده باشیم که *host*‌ها قابل اعتماد باشند، از کجا معلوم که پروتکل‌های مختلف (مثلا مدیریت کنترل ازدحام داده) را رعایت نکنند یا نکنند؟

- از کجا می‌توان فهمید که اپلیکیشن و سرویسی که ارائه می‌شود، اخلاقی و خوب است؟ چون هر کسی می‌تواند به راحتی، *application* را به اینترنت اضافه کرد.

از آنجایی که شبکه چندین کشور را در بر می‌گیرد، هیچ دولت خاصی نمی‌تواند مسئولیت تمام آن را بر عهده داشته باشد.

نیازمندی‌ها و چالش‌های جدید شبکه

با توجه به حجم انبوه داده، نشر و توزیع داده کار سخت تری خواهد شد. اینکه بعضی از *host*ها همزمان به چند تا *ISP* وصل باشند. یا به صورت سیار باشند و این موضوع در حال افزایش است. یا مثلاً بعضی از ارائه دهندگان اینترنت، همیشه به اینترنت وصل نباشند. همچنین تعداد زیاد *host*ها و کاربردهای *real time* از چالش‌های جدید خواهد بود.

بهبود در زمینه‌های امنیت، کارایی، اطمینان‌پذیری و همچنین تسهیل روند افزایش سرویس‌ها، *host*ها و تکنولوژی‌ها و اصلاحات فنی و سیاست‌گذاری درست، برای ارتقای اینترنت آتی لازم است.

مراجع

[1] “ch1-v8-slide-108-to-128,” <https://docs.google.com/drive/folders/>, 2020.